Linearizability and Serializability

two words that sound alike and mean different things

Watch the 4-min video

In one line: linearizability is a recency guarantee for single operations on one object (reads see the latest completed write, in real-time order); serializability is an isolation guarantee for transactions over many objects (they behave as if run one at a time, in some order); strict serializability gives both.

1 Two words, two ideas

Linearizability and **serializability** are constantly mixed up in interviews. One is about seeing the **latest value of one object**. The other is about **transactions over many objects** not interfering. A system can have either without the other.

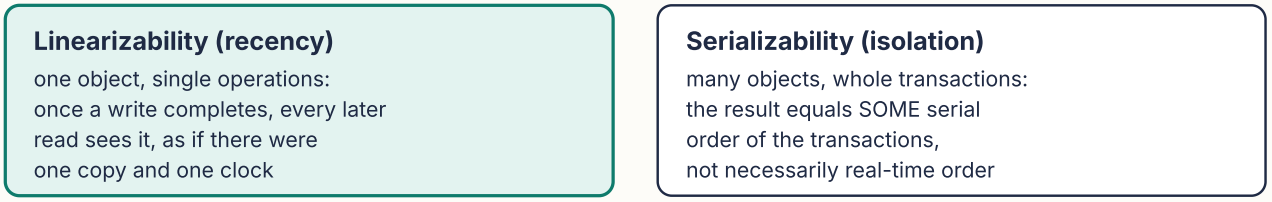


Figure 1. Linearizability is a consistency model for replicas; serializability is an isolation level for transactions.

2 Linearizability

A linearizable system behaves as if there were **one copy** of the data and every operation took effect atomically at some instant between its start and its end. So once a write has completed, any read that starts afterwards, from any client, must return that value or a newer one. It's what people mean by 'strong consistency' for a key-value store. You get it from single-leader systems that read from the leader (with care, lesson 9), consensus stores like etcd, and Spanner. Asynchronous replicas and most quorum setups don't provide it.

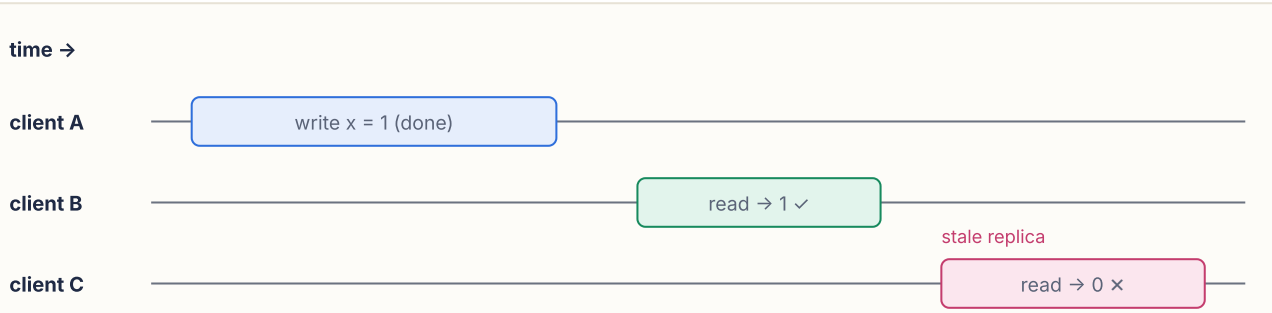


Figure 2. Not linearizable: C's read starts after B already saw x = 1, yet returns 0.

3 Serializability

A serializable database guarantees that concurrent transactions produce the same result as if they had run **one after another, in some order**. It prevents anomalies like lost updates, write skew and phantoms. It doesn't promise which order, or that the order matches real time: a transaction that committed earlier

could be ordered after one that started later. Implementations: two-phase locking, serializable snapshot isolation (Postgres SERIALIZABLE), or actual serial execution.

WORKED EXAMPLE: UNIQUE USERNAMES

Two people try to register the name `alice` at the same moment, on different servers. The check 'is alice taken?' and the insert happen in a transaction. Under **snapshot isolation**, both transactions read 'not taken' and both insert: write skew, two Alices. Under **serializability**, one of them aborts. In a replicated system, you also need the check to see the latest committed data: a linearizable read, or a unique constraint enforced by a single leader. The simplest robust answer: a unique index on `username`, enforced in one place.

4 Both at once

Strict serializability combines them: transactions are serializable, and the serial order respects real time. Spanner, CockroachDB (close to it) and FaunaDB offer this; it costs coordination, such as TrueTime commit waits or consensus round trips.

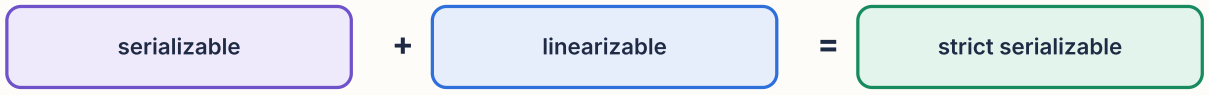


Figure 3. The strongest common guarantee, and the most expensive.

COMMON MISTAKES

- Saying 'serializable' when you mean reads see the latest data.
- Assuming the default isolation (often read committed) is serializable.
- Assuming quorum reads are linearizable.
- Checking uniqueness in application code without a constraint.

5 Key terms

Linearizability: single-object recency; real-time order

Strict serializability: serializable + real-time order

Write skew: two transactions read, then write different rows, breaking a rule

Serializability: transactions equal some serial order

Snapshot isolation: reads a consistent snapshot; allows write skew

6 Interview questions

Q1. What's the difference between linearizability and serializability? BASIC

Linearizability: single operations on one object appear instantaneous and in real-time order, so reads see the latest write. Serializability: transactions on many objects appear to run one at a time, in some order.

Q2. Is a single-leader database with async replicas linearizable? BASIC

Only for reads served by the leader. Reads from asynchronous replicas can be stale, which violates linearizability.

Q3. Two users claim the same username concurrently. How do you prevent duplicates? SCENARIO
Enforce a unique constraint in one strongly consistent place (a single leader or a consensus store); serializable transactions also work; app-level checks alone don't.

Q4. Which guarantee does a distributed lock service need? SCENARIO
Linearizability: every client must see the latest lock state, or two could think they hold the lock.

Q5. Can a system be serializable but not linearizable? TRICKY
Yes: transactions may be ordered in a valid serial order that doesn't match real time, for example reading from a slightly stale but consistent snapshot.

Q6. Why does CAP's 'C' refer to linearizability? TRICKY
CAP's proof uses linearizable reads and writes on a register: during a partition, you can't keep that recency guarantee and answer every request.

Next: SD-I-001 · Why Messaging backendroadmap.com